KUBERNETES K8S

1. K8S cluster architecture

Kubernetes cluster is a combination of **Master and worker nodes.** Master node we basically called it as a **brain of the cluster** and worker nodes would be like where our application is actually going to be run.. In Master node, we have four components which are **API server, Scheduler, etcd and controller manager**. In worker nodes, we have three components which are **kubelet, kube-poroxy, and pods.** Let us see all the components.

- **First from the master node,** we have an **API server** - it is the entry point of all the commands and it interacts with the client to the backend cluster, **Etcd-** it is like a key value distributed database that stores all the cluster state information. Then **Scheduler-**Its check the availability in nodes and where to run the pods based on the resources and policies.
  Controller-manager : it watches and monitors the cluster states, it ensures the desired state matches the reality. Eg: if we gave the replicas value 3 but only 2 pods are running it will create another pod.

- **On the other hand, we have workernode,** first we see the **kubelet** - it acts like an agent. It will be available in each working node it will create, delete, get the pods doing these kinds of activities. **Kubeproxy -** it controls the traffic to the **pods**, it helps to expose our application over the internet and

finally we have a **pods -** where our application is actually running inside the pod. These are the components where available in the cluster.

2. Replicaset : It ensures the pod counts based on the replicas value . for eg : if we give the replica value is 2 it always matches the pod counts also be 2. Once the pod crashes it automatically creates the pod.

3. Daemonset : It creates the pod based on the node count. For eg we had 2 workers nodes the daemonset automatically creates the 2 pods. For each working node it creates one pod like that.

4. Deployment : Deployment is also like a replica set but it has some strategies like Recreate, Rolling update and blue/Green strategies.

- **Recreate** - it will delete the existing pod and create the new pod for our application (down time will be high)
- **Rolling update** - It will create and then delete the existing pod. (down time will be very less).
- **Blue/Green Deployment** - It has 2 environments which are blue and green. Blue env is active running the application in live and green environment is in idle for testing the updation codes once it gets success the URL switches from blue to green. (0% downtime is here)

5. Namespace : Namespace is like an isolated workspace for different environment purposes like dev, qa, uat, prod. They will launch their pods inside their Namespace. We also set the permission to access the namespace in RBAC - Roles.

6. Pod : Pod is like a small deployable unit in k8s where our application containers are running.

7. services (cluster ip & nodeport & load balancer) : In k8s services are used to expose our application out to the world. There are 3 types of services which are cluster IP, Nodeport and Load balancer. Let us one by one, **Cluster ip :** It creates a static ip and is connected to a pod based upon the selector value, the main use of cluster ip is to communicate between one pod to another pod. It will be created inside the cluster. Then **Nodeport :** The nodeport will be created at the edge of the cluster and it will help to expose applications to the world. The nodeport range would be like 30000 to 32767. Then **Load balancer**, It will create a unique DNS name while creating the load balancer so the end user can easily access the application with domain name and it splits the traffic based on the target port.

8. Ingress : Ingress controller overcomes the load balancer, because it will create the out of the cluster same as a load balancer but we should set the rules here like path, so the traffic sends based on the paths to access the particular micro services.

9. Statefullset : Statefullset also likes a deployment but it has some key features like, The identity of the pod will not be changed while the pod gets restarted. Most cases it will be used in the database sectors and each pod keeps its own persistent volume and pod creates and destroys in a linear model like one after another one.

10. HPA : HPA stands for Horizontal Pod Autoscaler. This autoscaler creates the pod based on cpu and memory capacity. We should set the threshold based on that threshold limit. The HPA automatically creates the new pod till out max value and sends the traffic to the

new one, once the traffic is down it automatically destroys the pod and comes to min value.

11. VPA : VPA stands for Vertical Pod Autoscaler. Based on the traffic we will manually adjust the resource quota limit. It will create downtime compared to HPA.

12. resource (request & limit) : In Resource quota we can set request and limit value to access the cpu and memory capacity in container level. Request is like a min requirement capacity for containers and limit is like a max threshold for containers.

13. RBAC -> Role based access control : RBAC is one the features in k8s, we can set the permission and access to the users like IAM. Here we have RBAC components like user, role, rolebinding, cluster role and cluster role binding.

14. Role : In role, we set the permission in the name space level, which user is going to access the resource and that access the user will do some restricted actions like get,watch like that. If this role is attached to the user which is called **role-binding.**

15.Cluster Role : In Cluster role, we set the permission and access in the entire cluster level. If we attach the cluster role to the user which is called cluster role binding.

16. Scheduler types : In k8s we have different types of scheduler features which are Node selector, Affinity (Node affinity, pod affinity, anti pod affinity) and Taint & toleration.

17.Node selector : we should set the key value in the node level and we mentioned the nodeselector key value while creating the

pods. It checks the labels value, Once the label gets matched it launches the pod to that particular node.

18.Node Affinity : Node affinity also works like a node selector but here we have two rules which are hard and soft rules.

19.Hard rules :
RequiredDuringSchedulingIgnoresDuringExecution. It only launches the pod if labels exactly match, otherwise it doesn't launch the pod in other worker nodes.

20.Soft Rules:
PreferredDuringSchedulingIgnoresDuringExecution. It gives preference and checks the node label if it is matched it will launch the pod in the particular node . If it doesn't match it will launch the pod to other nodes.

21.Pod Affinity : Same like node affinity but in the pod affinity we should launch the pod based on the pod labels and we can mention the topology key which is the workernode host name. Here also we can set the soft and hard rules like required and preferred. Pod affinity is mainly used for microservices communication in the same worker node.

22.AntipodAffinity : It is the opposite version of pod affinity for eg: We have two microservices pods no need to be launched in the same worker node. That time we mentioned the anti-pod affinity.

23.Taint : yeah! Taint we should set in the node level like **kubectl taint nodes <nodename> key : value effects**

24.Toleration : Toleration working in pods we should mention the toleration while creating the pods with their effects.

25.Effects of taint and toleration: There are three types of effects which are, **Noschedule, PreferredNoSchedule, NoExecute.**

26.Noschedule : Taint and toleration labels do not match; it checks the effects. We mentioned no schedule there ; it doesn't launch the pod in other nodes.

27.PreferredNoSchedule : Taint and toleration labels do not match; it checks the effects. We mentioned PreferredNoSchedule on there and it launches the pod in other nodes.

28.NoExecute : If the Taint and toleration labels are matched; it checks the effects. We mentioned NoExecute there; it do not launch the pod in that particular node.

29.Operators in K8s : We have some set of operators in kubernetes which are IN, Exist, DoesNotExist, Equal.

- **IN :** we mentioned the key and some set of values in labels, it matches the key and launches the pod if any one of the values is matched.
- **Exist :** Once it matches the key it ignores the value, it will launch the pod once the key is matched.
- **DoesNotExist :**
- **Equal :** Here both the key and value should be matched. Then only it will launch the pod.

30.probes : Probes are to do the healthcheck in kubernetes Used by kubelet to know if the container is healthy and ready to serve the traffic. There is an different types of probes here which are :

- **Startuprobes :** It is used to check the container status is successfully started. Once the Readiness and Liveness probes get success this startup probes status will get success.
- **Livenessprobes :** It checks if the container is alive and running based on the period of time. If it fails it will kill the container and restart it.
- **Readiness probes :** This will check the container is Ready to serve the Traffic. If it fails, the pod will remove from the service endpoint.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
  - name: nginx
    image: nginx
    livenessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 10
    readinessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 3
      periodSeconds: 5
    startupProbe:
      httpGet:
        path: /
        port: 80
      failureThreshold: 30
      periodSeconds: 10
```

Copy code

31. Networking Why we use that : Basically by default, In cluster all the pods communicate with each other. One pod can easily access the other pods. So we should secure the traffic from one pod to another. For these kinds of restrictions we are using the Network Policies.

32. Network policy : A NetworkPolicy in Kubernetes is used to control network traffic (ingress & egress) between Pods, Namespaces, or external endpoints.
It works at layer 3/4 (IP + Port) level and requires a **CNI plugin** (e.g., Calico, Cilium, Weave). By default all traffic is allowed between Pods. When you apply a NetworkPolicy only allowed traffic (per rules) flows, everything else is blocked.

- **Allow traffic only from Pods with a specific label**

yaml Copy code

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend
  namespace: default
spec:
  podSelector:
    matchLabels:
      app: backend    # Apply policy to backend Pods
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend   # Allow traffic only from frontend Pods
```

- **Deny all egress traffic**

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all-egress
  namespace: default
spec:
  podSelector: {}
  policyTypes:
  - Egress
  egress: []
```

- **Allow Ingress from a Namespace**

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-namespace
spec:
  podSelector:
    matchLabels:
      app: db
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          team: frontend
```

33. CNI plugin : This plugin is basically called an agent in the cluster. It will assign the IP address while creating the pod. If this agent is not installed in the machine the Network policy wont work.

It's an open source plugin. There are different kinds of plugins available like Calico, Cilium, Weave like that.

34.Types of isolation : The isolation would be like we set the ingress rule, Egress rule and Both.

35.Cluster upgrade : Cluster upgradation is nothing but based on the requirement we will upgrade the version one by one from the worker node. Using the **cordon and Drain method.**

- **Cordon :** cordon is preventing the node, No new pod will launch on that workernode. we should mention like:

  Kubectl cordon <Nodename>

- **Drain :** Drain is delete all the pods in the workernode.

  kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data

Once the worker node is empty we do all the upgradation processes and then again start the pod, Once the uncordon commands run it creates the pod based on the deployment.

kubectl uncordon <node-name>

36. PDB (Pod Disruption Budget) : A PDB defines the minimum number of Pods that must be available. It overcomes the cluster upgradation whenever we use cordon and drain method. Definitely it will create downtime. Now I wont face the downtime in my application that's why we are using the PDB model. Whenever you use PDB method, if you Drain the node but not all the pods would be deleted at least min no of pods would be run. Ensures high

availability of critical workloads. Prevents Kubernetes from evicting too many Pods during maintenance.

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: myapp-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: myapp
```

kubectl get pdb | kubectl describe pdb myapp-pdb

37. Service mesh : It will encrypt the communication between service to service. Here we use the mTLS (Mutual Transport Layer Security) for encryption and One main advantage is when we launch the pod it automatically launches another one pod with that which is called as **side-car proxies.** We can integrate with the monitor tools like splunk, prometheus and watch the live dashboard in grafana to monitor the pod level activity.

38. Configmap : It is used to store the configuration data like key value pair, environment variables like that. Configmap is best practices to store Non-sensitive data.

39. Secret : It is used to store the sensitive data, whatever we give in the secrets it saves in encrypted format using the base64 algorithm. If others try to describe the secrets env variables the output shows in bytes size. Eg: 2bytes.

40.Helm chart : Helm chart is a package manager in k8s like apt, yum. It helps to reuse the yaml file. It works the same as a module in terraform. But here we create a **chart.yaml, values.yaml, template → Deployment.yaml, service.yaml.** Based on the file structure we can split our yaml code. When our we run the deployment file we just directly call the values from values.yaml and apiversion, kinds from chart.yaml

We can initialize the helm in our directory which **my chart** using helm install myrelease ./mychart

```bash
# Basic upgrade
helm upgrade myrelease ./mychart

# Upgrade with values file
helm upgrade myrelease ./mychart -f values.yaml

# Override values from CLI
helm upgrade myrelease ./mychart --set replicaCount=3

# Upgrade and install if not installed
helm upgrade --install myrelease ./mychart

# Check status after upgrade
helm status myrelease
```

41. Service Account: From our k8s, Pod is to access the different cloud platform resources with the help of service account, This will use the OIDC (OpenIDConnect) which is mandatory to access the other resources. Steps to attach the permission to the pod.

- First, We set up the roles in IAM for particular resources in inlinepolicy.
- Second, We create the service account in cluster with mention the Role arn which we have created in IAM.

- Then, Attach the service account to the pod, Now pod can access the resources based on the role permission.